

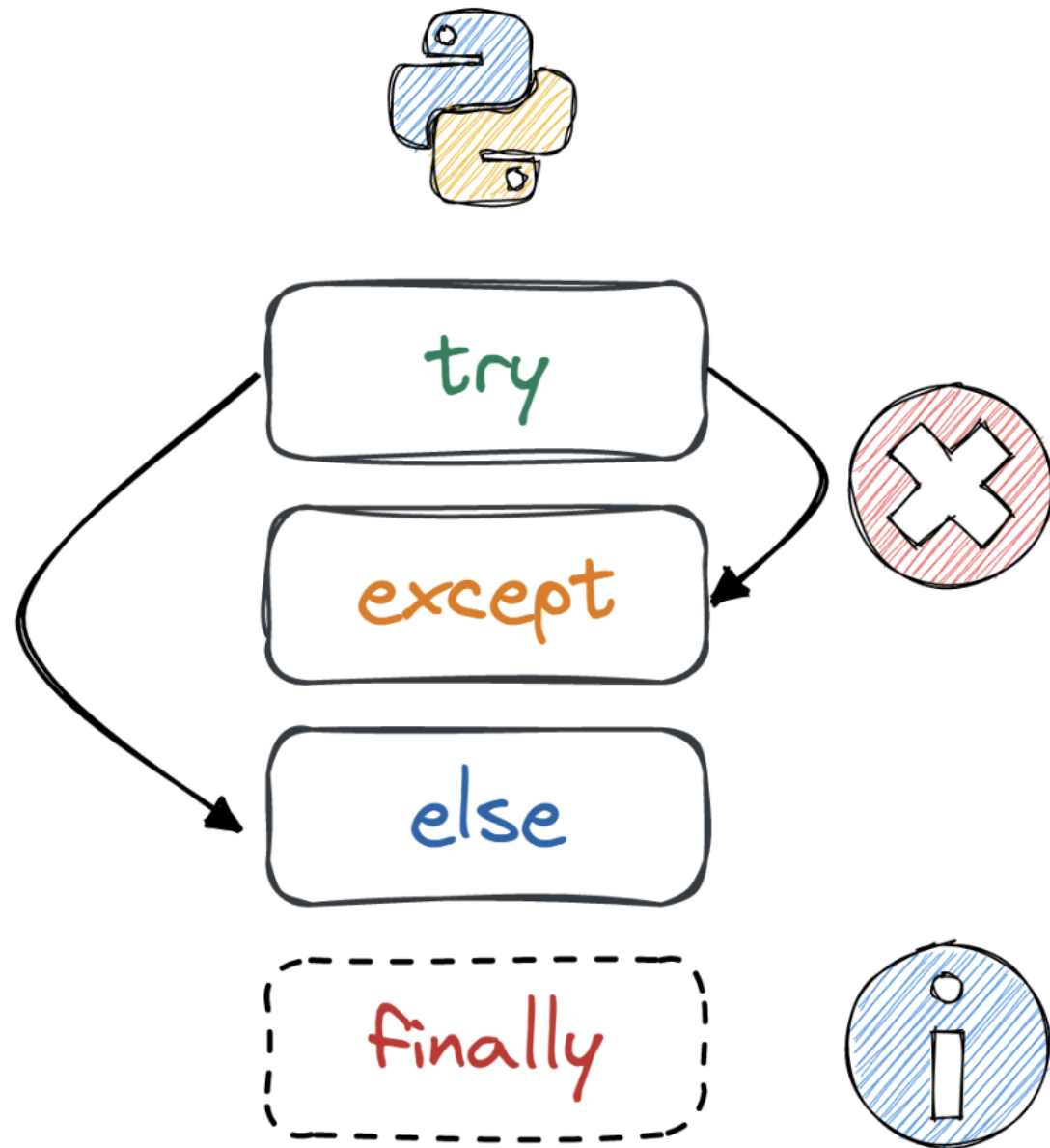


Exceptions handling in Python

Exception handling is a crucial part of any programming language. It helps us to write robust and reliable programs that can gracefully handle errors and unexpected situations. In Python, there are several built-in mechanisms for exception handling, making it easy to write code that is both functional and resilient.



by Prof. M. Iqbal Bhat



Anything that can go
wrong, will go wrong.

Murphy's law

```
import math
```

```
x = float(input("Enter x: "))
```

```
y = math.sqrt(x)
```

```
print("The square root of", x, "equals to", y)
```

- There are at least two possible ways it can "go wrong":
 1. As a user is able to enter a completely arbitrary string of characters, there is no guarantee that the string can be converted into a float value – this is the first vulnerability of the code;
 2. the second is that the sqrt() function fails if it gets a negative argument.

Exception Handling in Python:

Python provides effective tools that allow you to observe exceptions, identify them and handle them efficiently. This is possible due to the fact that all potential exceptions have their **unambiguous names**, so you can categorize them and react appropriately.

You know some exception names already.

```
ValueError : math domain error
```

The word highlighted above is just the exception name.

Exception Handling in Python:

How do you handle exceptions? The word try is the key to the solution.

The recipe for success is as follows:

- first, you have to try to do something;
- next, you have to check whether everything went well.

```
first_number = int(input("Enter the first number: "))
second_number = int(input("Enter the second number: "))

try:
    print(first_number / second_number)
except:
    print("This operation cannot be done.")

print("THE END.")
```

Exception Handling in Python:

```
try:
    :
    :
except:
    :
    :
```

- the **try** keyword begins a block of the code which may or may not be performing correctly;
- next, Python tries to perform the risky action; if it fails, an exception is raised and Python starts to look for a solution;
- the **except** keyword starts a piece of code which will be executed if anything inside the try block goes wrong – if an exception is raised inside a previous try block, it will fail here, so the code located after the **except** keyword should provide an adequate reaction to the raised exception;
- returning to the previous nesting level ends the try-except section.

Exception Handling in Python:

```
try:  
    print("1")  
    x = 1 / 0  
    print("2")  
except:  
    print("Oh dear, something went wrong...")  
  
print("3")
```

Exception Handling in Python:

```
1 try:
2     x = int(input("Enter any number: "))
3     y = 1 / x
4 except:
5     print("Ooops!, something went wrong...")
6
7 print("Last Print Statement")
```

There are two possible causes of the exception in above program:

1. non-integer data entered by the user;
2. an integer value equal to 0 assigned to the x variable.

But console says nothing about the reason of the Exception.

Exception Handling in Python:

```
1 try:
2     x = int(input("Enter any number: "))
3     y = 1 / x
4 except exception1:
5     print("Exception1 occurred")
6 except exception2:
7     print("Exception2 occurred")
8
9 print("Last Print Statement")
```

Technically, there are two ways to solve the issue:

1. build two consecutive try-except blocks, one for each possible exception reason (easy, but will cause unfavorable code growth)
2. use a more advanced variant of the instruction.

Exception Handling in Python:

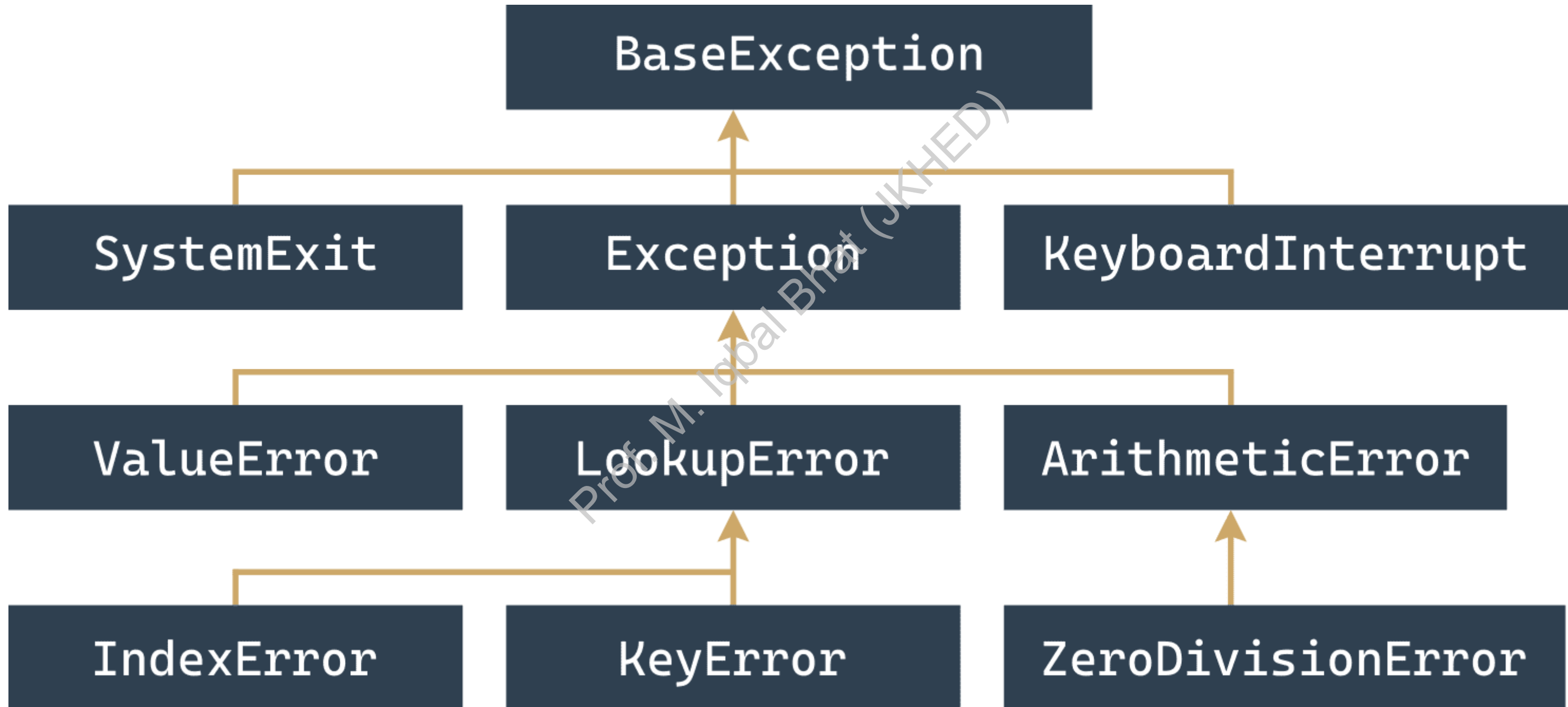
```
1  try:
2      x = int(input("Enter any number: "))
3      y = 1 / x
4      print(y)
5  except ZeroDivisionError:
6      print("You cannot divide by zero, sorry.")
7  except ValueError:
8      print("You must enter an integer value.")
9  except:
10     print("Ooops!, something went wrong...")
11 print("Last Print Statement")
```

Exception Handling in Python:

Points to Remember:

1. the except branches are searched in the same order in which they appear in the code;
2. you must not use more than one except branch with a certain exception name;
3. the number of different except branches is arbitrary – the only condition is that if you use try, you must put at least one except (named or not) after it;
4. the except keyword must not be used without a preceding try;
5. if any of the except branches is executed, no other branches will be visited;
6. if none of the specified except branches matches the raised exception, the exception remains unhandled.
7. if an unnamed except branch exists (one without an exception name), it has to be specified as the last.

Exception Hierarchy:



Famous Exception:

ArithmeticError

Location: `BaseException ← Exception ← ArithmeticError`

Description: an abstract exception including all exceptions caused by arithmetic operations like zero division or an argument's invalid domain

AssertionError

Location: `BaseException ← Exception ← AssertionError`

Description: a concrete exception raised by the `assert` instruction when its argument evaluates to `False`, `None`, `0`, or an empty string

```
from math import tan, radians
angle = int(input('Enter integral angle in degrees: '))

# We must be sure that angle != 90 + k * 180
assert angle % 180 != 90
print(tan(radians(angle)))
```

Famous Exception:

IndexError

Location: `BaseException ← Exception ← LookupError ← IndexError`

Description: a concrete exception raised when you try to access a non-existent sequence's element (e.g., a list's element)

Code:

```
# The code shows an extravagant way  
# of leaving the loop.
```

```
the_list = [1, 2, 3, 4, 5]  
ix = 0  
do_it = True  
  
while do_it:  
    try:  
        print(the_list[ix])  
        ix += 1  
    except IndexError:  
        do_it = False  
  
print('Done')
```

Famous Exception:

KeyboardInterrupt

Location: `BaseException ← KeyboardInterrupt`

Description: a concrete exception raised when the user uses a keyboard shortcut designed to terminate a program's execution (*Ctrl-C* in most OSs); if handling this exception doesn't lead to program termination, the program continues its execution.

Note: this exception is not derived from the `Exception` class. Run the program in IDLE.

Code:

```
# This code cannot be terminated
# by pressing Ctrl-C.

from time import sleep

seconds = 0

while True:
    try:
        print(seconds)
        seconds += 1
        sleep(1)
    except KeyboardInterrupt:
        print("Don't do that!")
```

Famous Exception:

OverflowError

Location: `BaseException ← Exception ← ArithmeticError ← OverflowError`

Description: a concrete exception raised when an operation produces a number too big to be successfully stored

Code:

```
# The code prints subsequent
# values of exp(k), k = 1, 2, 4, 8, 16, ...

from math import exp

ex = 1

try:
    while True:
        print(exp(ex))
        ex *= 2
except OverflowError:
    print('The number is too big.')
```


Famous Exception:

ImportError

Location: BaseException ← Exception ← StandardError ← ImportError

Description: a concrete exception raised when an import operation fails

Code:

```
># One of these imports will fail - which one?
```

```
try:
```

```
    import math
```

```
    import time
```

```
    import abracadabra:
```

```
except:
```

```
    print('One of your imports has failed.')
```

Famous Exception:

KeyError

Location: BaseException ← Exception ← LookupError ← KeyError

Description: a concrete exception raised when you try to access a non-existent element in a collection (e.g., a dictionary's element)

Code:

```
# How to abuse the dictionary
# and how to deal with it?

dictionary = {'a': 'b', 'b': 'c', 'c': 'd'}
ch = 'a'

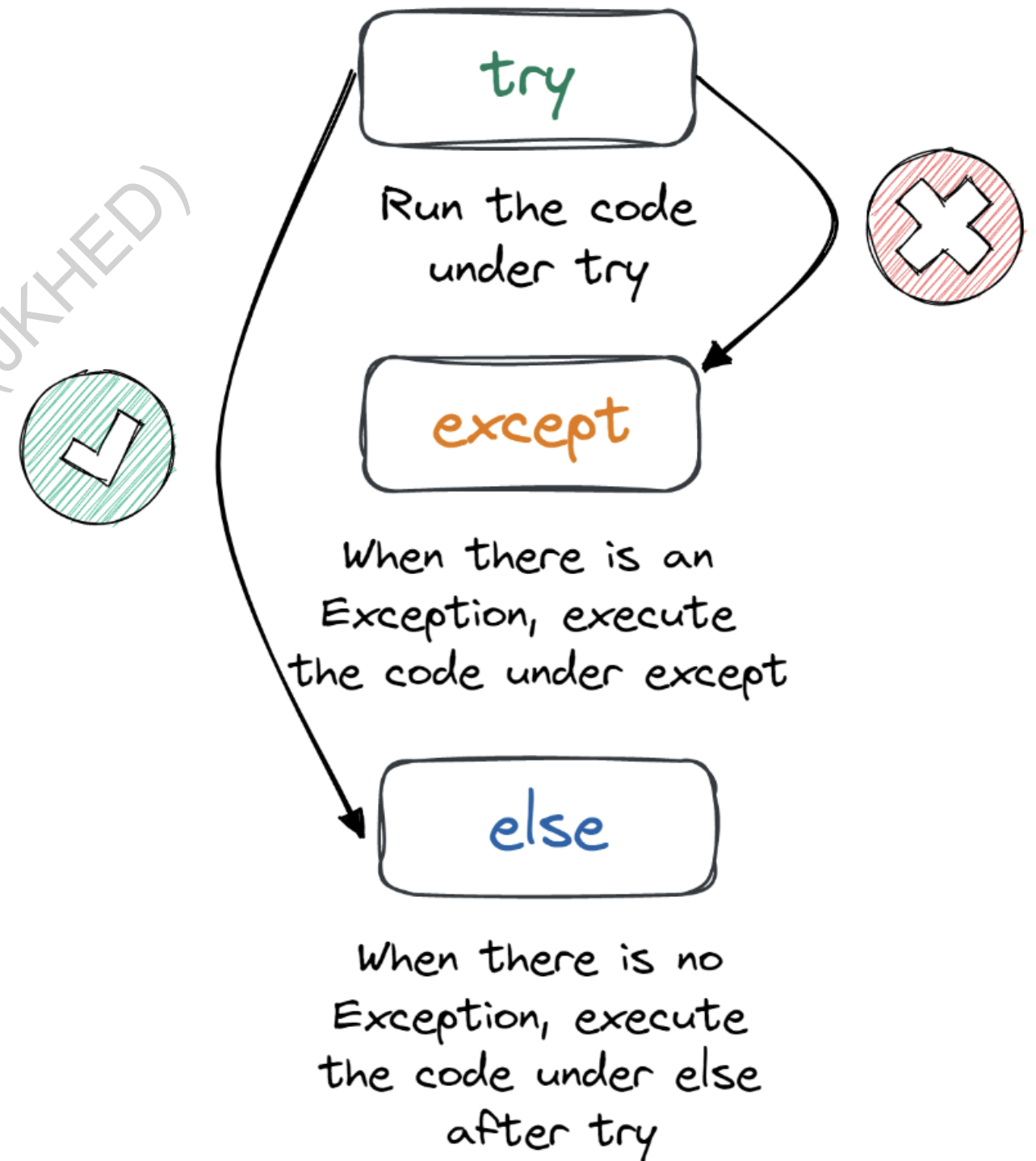
try:
    while True:
        ch = dictionary[ch]
        print(ch)
except KeyError:
    print('No such key:', ch)
```

<https://docs.python.org/3.6/library/exceptions.html>

Try with Else Clause:

try-else Clause:

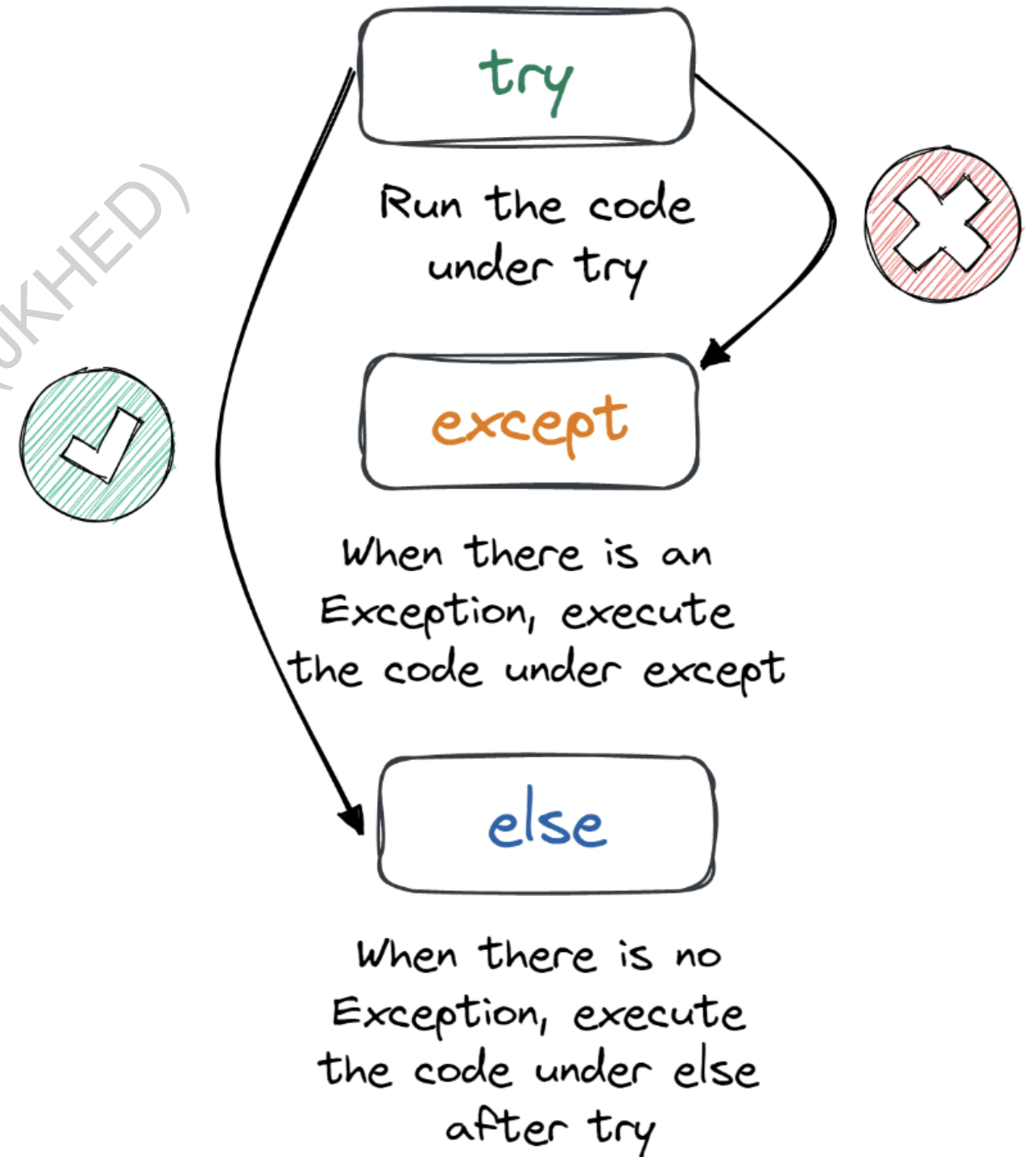
1. When the `try` statement does not raise an exception, code enters into the `else` block.
2. It is the remedy or a fallback option when you expect a part of your script will produce an exception.
3. It is generally used in a brief setup or verification section where you don't want certain errors to hide.



Try with Else Clause:

```
1 try:
2     result = 1/3
3 except ZeroDivisionError as err:
4     print(err)
5 else:
6     print(f"Your answer is {result}")
```

```
1 x = [5,8,9,13]
2
3 def find_nth_value(x,n):
4     try:
5         result = x[n]
6     except IndexError as err:
7         print(err)
8     else:
9         print("Your answer is ", result)
```



Try with finally Clause:

finally Clause:

1. The `finally` keyword in the try-except block is always executed, irrespective of whether there is an exception or not.
2. In simple words, the `finally` block of code is run after the try, except, the else block is final.
3. It is quite useful in cleaning up resources and closing the object, especially closing the files.

try

except

else

finally

Always run the code
under finally.

Try with finally Clause:

```
1 def divide(x, y):  
2     try:  
3         result = x / y  
4     except ZeroDivisionError:  
5         print("Please change 'y' argument to non-zero value")  
6     except:  
7         print("Something went wrong")  
8     else:  
9         print(f"Your answer is {result}")  
10    finally:  
11        print("This statement is always executed")
```

try

except

else

finally

Always run the code
under finally.

Raising Exceptions in Python:

raise exceptions:

1. As a Python developer, you have the option to throw an exception if certain conditions are met. It allows you to interrupt the program based on your requirement.
2. To throw an exception, we have to use the `raise` keyword followed by an exception name.

```
value = int(input("Enter the value"))
if value > 1_000:
    # raise the ValueError
    raise ValueError("Please add a value lower than 1,000")
else:
    print("Congratulations! You are the winner!!")
```